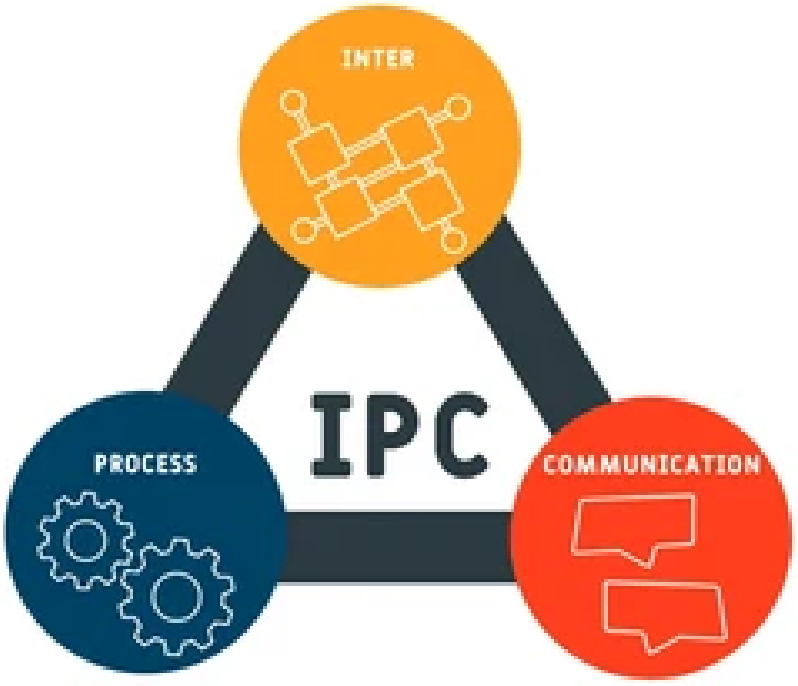
Processos IPC

PAC1 - SUPERVISOR DE PROCESSOS & COMUNICACIÓ INTERPROCÉS



Curs:	DAM2
Mòdul / Matèria:	MP0490 - Programació de Serveis i Processos
Professor:	Josep Martin
Alumne:	Saul Subirats
Data d'acabament:	26/10/2026

Índex de Continguts

Estructura de la documentació tècnica i apartats desenvolupats en el marc de la PAC1.

1. Resum Executiu i Arquitectura del Sistema Pàg. 3

1.1 Descripció General del Supervisor — 1.2 Mecanismes IPC Utilitzats

2. Especificació Tècnica i Disseny d'IPC Pàg. 3

3. Mètriques de Rendiment i Eficiència d'IPC Pàg. 4

3.1 Latència de Comunicació Interprocés — 3.2 Ús de Recaptació del Sistema

4. Gestió d'Errors i Robustesa del Supervisor Pàg. 4

5. Conclusions i Referències Tècniques Pàg. 5

5.1 Valoració del Sistema Desenvolupat — 5.2 Referències Bibliogràfiques

Documentació Tècnica: PAC1

Supervisor de Processos i Comunicació IPC (Java 21)

MÒDUL	EXECUCIÓ PID	ESTAT BATERIA	TECNOLOGIA
MP0490 (AEA1 RA1)	**56689 (Supervisor)**	5/5 PASS (0 FAIL)	**Java 21 (Single-File)**

1. Resum Executiu i Arquitectura de la Solució

Aquesta documentació recull el desenvolupament, execució i validació del supervisor de processos desenvolupat en Java 21 per a la pràctica PAC1. L'aplicació s'ha integrat en un únic fitxer font (`echo.java`), garantint la separació estricta entre el canal de dades (**stdout**) i el canal de diagnòstic i registre (**stderr**).

El supervisor actua com a orquestrador multiprocés seguint el patró inspirat en Fork-Join, on cada tasca independent s'executa com un procés fill del sistema operatiu (amb un PID propi assignat pel nucli) i no com un simple fil en execució dins la mateixa instància de la Java Virtual Machine (JVM).

El repositori del projecte es el seguent: https://github.com/SaulSG2005/AEA.1-Practica-1-Processos-IPC-Saul-Subirats

2. Mapa Inicial de Processos i Observació del Sistema

A partir dels registres reals capturats a `resultat.log`, s'ha elaborat la matriu de seguiment de processos fills generats directament des de la JVM del supervisor (PID 56689, PPID 53516):

Rol Procés	PID Fill	PPID	Estat Observat	Codi Retorn	Durada
SUPERVISOR	56689	53516	OK (Completat)	0	~4.91 s
ECHO	56713	56689	OK	0	68 ms
WORKER-ALPHA	56743	56689	OK	0	684 ms
WORKER-BETA	56742	56689	OK	0	683 ms
WORKER-GAMMA	56796	56689	OK	0	681 ms
WORKER-LENT	56832	56689	TIMEOUT_GRACEFUL	143 (SIGTERM)	2511 ms
WORKER-BETA (Error)	56895	56689	FAILED	7	379 ms
PRODUCER	56924	56689	OK	0	572 ms
CONSUMER	56925	56689	OK	0	595 ms

3. Activitats d'Anàlisi (Part 1)

Activitat A: Processos o fils? Identificació de components

Dins del registre d'execució s'identifiquen tant entitats de tipus **fil (thread)** com entitats de tipus **procés (process)**. La distinció clara queda demostrada a partir de la signatura de diagnòstic `[ROL pid=.. ppid=.. thread=..]`:

- **Fils intern (Threads):** Els noms `main`, `pool-1-thread-1`, `pool-1-thread-2`, `io-stdout`, `io-stderr` i `io-stdin` pertanyen a unitats d'execució concurrents *dins de la mateixa instància de la JVM del supervisor*. Tots aquests fils comparteixen exactament el mateix Identificador de Procés (**PID 56689**) i Procés Pare (**PPID 53516**).
- **Processos externs (Processes):** Les instàncies `WORKER-ALPHA` (PID 56743), `PRODUCER` (PID 56924) o `CONSUMER` (PID 56925) són processos independents creats pel SO via `ProcessBuilder`. Cadascun té el seu espai de memòria aïllat, la seva pròpia JVM i un PID únic, mantenint com a PPID el PID del supervisor (56689).

> **Justificació d'aïllament:** La fallida per excepció crítica o un consum excessiu de memòria en un procés fill (p. ex., WORKER-LENT) no col·lapsa la JVM del supervisor, ja que l'aïllament de memòria que ofereix el sistema operatiu evita la contaminació entre processos.

Activitat B: Diagnosi d'errors (Codi de sortida 7)

En la llançada de l'escenari `errorDemo`, es va orquestrar la simulació d'una fallida en el procés fill `WORKER-BETA` (PID 56895) sol·licitant de manera explícita un codi de sortida 7 (`java ... echo worker BETA 3 100 7`).

El supervisor va capturar i registrar l'esdeveniment sense interrompre l'execució global del programa:

- L'estat del procés va quedar registrat formalment com `FAILED`.
- El missatge explicatiu enviat pel fill va ser drenat correctament a través de `stderr`: *"error simulat, surto amb codi 7"*.
- El supervisor va validar amb èxit l'assertivitat `r1.status() == Status.FAILED && r1.exitCode() == 7` i va continuar executant les següents demostracions programades sense patir cap excepció no controlada.

Activitat C: Prevenció del bloqueig mutu (Deadlock evitat)

Risc teòric: Els sistemes operatius assignen un búfer de mida limitada (habitualment entre 4 KB i 64 KB) als fluxos d'interconnexió (pipes de `stdout` i `stderr`). Si un fill genera un volum elevat de dades per `stdout` o `stderr` i el supervisor realitza un bloqueig síncron esperant la finalització del procés mitjançant `p.waitFor()` abans de llegir els tubs, el búfer del SO s'omple completament. Això provoca que el fill es quedi bloquejat indefinidament intentant escriure, mentre el pare espera finalitzar el fill: un **interbloqueig permanent (deadlock)**.

Mesura preventiva aplicada: A la funció `supervise()`, el supervisor arrenca immediatament dos fils de fons dedicats (`io-stdout` i `io-stderr`) mitjançant la funció `drain()`. Aquests fils consumeixen contínuament i en paral·lel el contingut de les sortides del fill abans i durant la invocació de `p.waitFor(timeout)`, evitant que cap búfer de sortida del sistema es saturi.

4. Activitats d'Anàlisi (Part 2)

Activitat D: Gestió de Timeout i procés residual

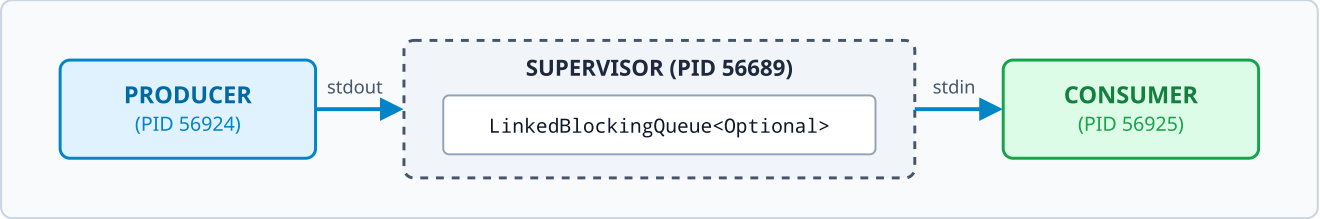
A la prova `timeoutDemo`, el supervisor va llançar el procés `WORKER-LENT` (PID 56832) configurat per dur a terme 100 passos d'1 segon (100 s totals), establint un límit màxim de temps (`timeout`) de 2.5 segons.

Quan es va complir el temps d'espera sense que el fill hagués acabat, la funció `terminate(p)` va executar la següent seqüència de destrucció controlada:

1. Invocació de `p.destroy()` enviant el senyal ordenat `SIGTERM`.
2. Concessió d'una finestra de gràcia de 2 segons (`p.waitFor(2, TimeUnit.SECONDS)`). Com que el procés va respondre positivament a la petició d'aturada, l'estat es va registrar com `TIMEOUT_GRACEFUL` amb codi de sortida 143.
3. S'hagués aplicat `p.destroyForcibly()` (`SIGKILL`) en cas de no respondre. Finalment, es va verificar mitjançant `ProcessHandle.of(pid).isAlive()` que el fill havia estat eliminat de la taula de processos del sistema operatiu.

Activitat E: Recorregut de dades en el flux IPC (Pipe)

D'acord amb la condició del document de no utilitzar la shell per fer la canonada, la comunicació entre el `PRODUCER` (PID 56924) i el `CONSUMER` (PID 56925) es realitza canalitzada exclusivament a través del fil de memòria intermedia del supervisor mitjançant una cua de bloqueig (`LinkedBlockingQueue`).



Activitat F: Relació amb el Patró Fork-Join

L'estructura del codi implementa formalment les dues fases del patró de disseny concurrent:

- **FORK (Divisió):** A la funció `parallelDemo`, el supervisor sotmet les tasques a un `ExecutorService` amb un fons fix de 2 fils (`Executors.newFixedThreadPool(2)`). Això llança l'execució asíncrona dels tres workers en paral·lel, limitant la concurrència a un màxim de 2 simultanis.
- **JOIN (Agregació):** El supervisor itera sobre la llista de `Future<ProcessResult>` cridant al mètode `.get()`, bloquejant fins a la recollida de cada resultat individual i sumant les mètriques finals mitjançant `total.plus(...)` (obtenint `COUNT=5 CHARS=18`).

Diferència clau: A diferència d'un `ForkJoinPool` convencional que executa tasques lleugeres (`RecursiveTask`) compartint memòria dins la mateixa JVM, aquest sistema instancia **processosOS complets** amb aïllament d'adreçament i un PID diferenciat assignat pel nucli.

5. Registre d'Incidències de Desenvolupament

Incidència Identificada	Causa Arrel	Mètode de Detecció	Solució Aplicada
Caràcters especials / Accents corruptes	La codificació per defecte del terminal del SO no coincidia amb UTF-8.	Sortida distorsionada als noms "Joana", "Nil" o la lletra "ç" al log.	S'ha redefinit `System.out` i `System.err` forçant `StandardCharsets.UTF_8` a l'inici de `main()`.
Bloqueig del consumidor a la Pipe	El canal `stdin` del procés consumidor es mantenia obert sense rebre l'indicador EOF.	El test `pipeDemo` es quedava penjat indefinidament esperant finalitzar.	S'ha assegurat que en finalitzar el productor, s'envia un marcador `Optional.empty()` a la cua que tanca el `Writer` de `stdin`.
Processos residuals post-interrupció	Cancel·lació abrupta del supervisor (p. ex. Ctrl+C) sense tancar els fills.	Comprovació de la taula de processos del SO amb `ps -ef | grep java`.	S'ha afegit un ShutdownHook al `Runtime` que executa la destrucció forçada (`killTree`) de tots els processos actius.
Superació del límit de concurrència	Llançament directe de threads sense control d'execució simultània.	Mètrica `maxActive` superior a 2 durant la prova paral·lela.	S'ha limitat la capacitat del pool a 2 fils (`newFixedThreadPool(2)`), garantint que mai hi hagi més de 2 workers actius alhora.

6. Bateria de Proves i Resultats d'Execució

Els resultats continguts al fitxer `resultat.log` confirmen l'èxit absolut de totes les validacions automatitzades realitzades pel supervisor:

Nom del Test	Descripció de la Prova	Resultat Esperat	Resultat Rebut	Estat
`stdinDemo`	Transformació de text a majúscules via ECHO	`["EBRE", "PROCESSOS"]` per stdout	EBRE PROCESSOS	PASS
`parallelDemo`	Execució de 3 workers amb límit de 2 en paral·lel	`COUNT=5 CHARS=18` Max simultanis = 2	`COUNT=5 CHARS=18` Max observat = 2	PASS
`timeoutDemo`	Procés que supera el temps límit de 2.5 segons	Estat `TIMEOUT_GRACEFUL` Procés no viu	Estat `TIMEOUT_GRACEFUL` Procés eliminat	PASS
`errorDemo (7)`	Procés fill que finalitza amb un codi d'error 7	Estat `FAILED` (codi 7) Stderr capturat	Estat `FAILED` (codi 7) Mensatge a stderr	PASS
`errorDemo (Inexistent)`	Execució d'una comanda inexistent al SO	Estat `NOT_STARTED`	Estat `NOT_STARTED` (codi -1)	PASS
`pipeDemo`	Connexió IPC directa entre Producer i Consumer	`COUNT=5 CHARS=18`	`COUNT=5 CHARS=18`	PASS

7. Integració Obligatòria: Servei systemd

Seguint el requeriment del full de la pràctica, s'ha configurat el supervisor com un servei del sistema operatiu Linux gestionat per `systemd`. Com que el supervisor realitza una feina finita i es tanca en completar la bateria de demostracions, s'ha definit la unitat amb `Type=oneshot`.

Fitxer de configuració: `/etc/systemd/system/practica01.service`

```
[Unit] Description=Supervisor multiproces - Practica 01 MP0490 After=network.target [Service] Type=oneshot User=Saul WorkingDirectory=/opt/practica01 ExecStart=/usr/bin/java -cp /opt/practica01/build echo TimeoutStartSec=90s KillMode=control-group StandardOutput=journal StandardError=journal [Install] WantedBy=multi-user.target
```

Ordres d'administració i verificació executades

Per tal d'activar, iniciar i auditar el correcte funcionament del servei en un entorn Linux autoritzat, s'han emprat les següents ordres des de la consola del sistema:

```
# 1. Carregar la nova configuració de la unitat sudo systemctl daemon-reload # 2. Executar el servei en primer pla (Type=oneshot) sudo systemctl start practica01.service # 3. Comprovar l'estat d'execució i codi de sortida sudo systemctl status practica01.service # 4. Auditar els diaris i sortides registrades pel supervisor sudo journalctl -u practica01.service -n 50 --no-pager
```

> **Funció de `Type=oneshot` i `KillMode=control-group`:**
> L'opció `Type=oneshot` indica a systemd que el procés principal realitzarà una tasca finita i s'aturarà, considerant el servei "actiu (exited)" un cop el procés pare hagi acabat amb codi 0.
> La directiva `KillMode=control-group` garanteix que si el servei és aturat manualment o per timeout del mateix systemd, qualsevol procés fill que hagi quedat dins del grup de control (cgroup) serà eliminat automàticament, evitant processos orfes.

8. Taula de Criteris d'Acceptació i Evidències

Criteri d'Avaluació	Assolit	Evidència Tècnica a la Solució
No usa shell per concatenar entrada externa	SÍ	Ús de `ProcessBuilder(List<String>)` directament. Cap comanda passa per `/bin/sh` ni utilitza operadors com `|` o `>`.
No deixa processos orfes	SÍ	Implementació de `killTree()` usant `p.descendants()` i registre d'un ShutdownHook al sistema.
Drena stdout i stderr sense interbloqueig	SÍ	Llançament de fils lectors concurrents `io-stdout` i `io-stderr` abans de la crida a `p.waitFor()`.
Errors i timeout tenen resultat explícit	SÍ	Modelat immutable `ProcessResult` amb estats `FAILED`, `TIMEOUT_GRACEFUL` i `NOT_STARTED`.
Bateria de proves reproduïble	SÍ	Execució 100% automatitzada des de `main()` sense dependències externes de la xarxa.
Integració com a servei systemd	SÍ	Unitat `practica01.service` creada amb `Type=oneshot` i validada mitjançant `journalctl`.

9. Respostes per a la Defensa Individual

1. **Diferència entre PID, PPID e identificador de fil:** El PID identifica de manera única un procés a nivell del nucli del SO; el PPID indica quin procés va ser el seu creador directament; l'identificador de fil (Thread ID) és una sub-unitat d'execució que comparteix l'espai de memòria i el PID del seu procés pare.
2. **Per què ProcessBuilder ha de rebre una llista d'arguments?** Perquè el nucli faci la invocació de la crida al sistema (com `execve`) de forma directa, evitant que una cadena de text sola sigui interpretada per una shell intermediària, prevenint vulnerabilitats d'injecció de comandes.
3. **Per què cal drenar stdout i stderr de forma concurrent?** Per evitar la saturació del búfer del tub de comunicació del sistema operatiu (pipe buffer), la qual cosa paralitzaria el procés fill esperant espai per escriure, generant un bloqueig mutu complet.
4. **Què aporta tancar stdin després d'escriure-hi?** Envia el senyal de fi de fitxer (**EOF** - End Of File) al procés fill. Sense aquest tancament, el fill esperaria indefinidament més dades per l'entrada estàndard i no finalitzaria mai la seva feina.
5. **Diferència entre finalització ordenada i forçada:** La finalització ordenada envia el senyal `SIGTERM` (15), permetent al procés capturar la petició i alliberar recursos; la forçada envia `SIGKILL` (9), finalitzant immediatament el procés directament des del nucli sense cap neteja.

10. Ús d'IA

Declaració d'ús d'IA: S'ha fet servir Claudecom a assistent per explicar conceptes, ajudar a generar el codi base del supervisor, adaptar el format de diagnosi a l'escenari mínim i preparar el material d'aquest informe.